

Graphs

5/23/25

Administrivia

- HW 10 (Create a design problem) due Tuesday at noon
 - We'll go over them on Wednesday (5/28), along with any other questions
- No lab on last day of classes (5/29); I'll be there if you have questions
- Final a week from Monday (6/2) in the afternoon (1:30-4:30pm)
 - Open notes, closed friends and internet
 - Comprehensive, but weighted toward material since the midterm

Finding minimum

```
int min = nums[0];  
for(int i=1; i < nums.length; i++)  
    if(nums[i] < min)  
        min = nums[i];
```

Finding second smallest

```
int first = nums[0];           //smallest so far
int second = nums[1];          //second smallest so far
if(first > second)
    swap first and second;
for(int i=2; i < nums.length; i++) {
    if(nums[i] < second) {
        second = nums[i];
        if(second < first)
            swap first and second;
    }
}
```

Finding k^{th} smallest

```
int[] smallest = new int[k];           //k smallest values so far
...
for(int i=k; i < nums.length; i++) {
    if(nums[i] < smallest[k-1]) {
        int j = k-1;
        while((j > 0) && (nums[i] < smallest[j-1])) {
            smallest[j] = smallest[j-1];
            j--;
        }
        smallest[j] = nums[i];
    }
}
```

Finding k^{th} smallest

```
int[] smallest = new int[k];           //k smallest values so far
```

```
...
```

```
for(int i=k; i < nums.length; i++) {
```

```
    if(nums[i] < smallest[k-1]) {
```

```
        int j = k-1;
```

```
        while((j > 0) && (nums[i] < smallest[j-1])) {
```

```
            smallest[j] = smallest[j-1];
```

```
            j--;
```

```
        }
```

```
        smallest[j] = nums[i];
```

```
    }
```

```
}
```

What is the running time of this?

A. $O(n)$

B. $O(n+k)$

C. $O(n \log k)$

D. $O(n^2)$

E. None of the above

Finding k^{th} smallest

```
int[] smallest = new int[k];           //k smallest values so far
```

```
...
```

```
for(int i=k; i < nums.length; i++) {
```

```
    if(nums[i] < smallest[k-1]) {
```

```
        int j = k-1;
```

```
        while((j > 0) && (nums[i] < smallest[j-1])) {
```

```
            smallest[j] = smallest[j-1];
```

```
            j--;
```

```
        }
```

```
        smallest[j] = nums[i];
```

```
    }
```

```
}
```

What is the running time of this?

A. $O(n)$

B. $O(n+k)$

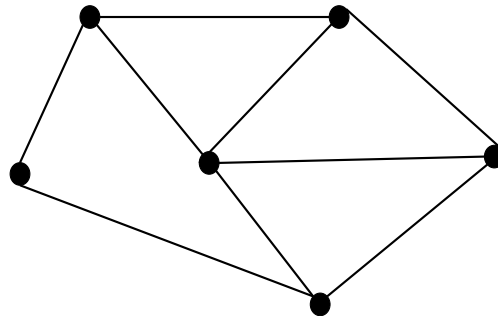
C. $O(n \log k)$

D. $O(n^2)$

E. None of the above $O(nk)$

Graphs

- (V, E) = set of vertices V and set of edges E
 - Edges can be directed or undirected
 - Vertices and edges may have extra data attached to them



Applications

- Geographic information
- Computer networks
- Web graph, citation graph
- Facebook graph
- Precedence relationships
- State transitions

Depth-First Search (DFS)

```
mark origin as visited;
vertexStack.push(origin);
while(!vertexStack.isEmpty()) {
    topVertex = vertexStack.peek();
    if(topVertex has an unvisited neighbor) {
        nextNeighbor = next unvisited neighbor of topVertex;
        mark nextNeighbor as visited;
        vertexStack.push(nextNeighbor);
    } else
        vertexStack.pop();
}
```

Recursive DFS

```
DFS(Vertex v) {  
    while(v has another neighbor) {  
        u = next neighbor of v;  
        if(u is unmarked) {  
            mark u;  
            DFS(u);  
        }  
    }  
}
```

Breadth-First Search (BFS)

```
mark origin as visited;
vertexQueue.enqueue(origin);
while(!vertexQueue.isEmpty()) {
    frontVertex = vertexQueue.dequeue();
    while(frontVertex has a neighbor) {
        nextNeighbor = next neighbor of frontVertex;
        if(nextNeighbor is not visited) {
            mark nextNeighbor as visited;
            vertexQueue.enqueue(nextNeighbor);
        }
    }
}
```

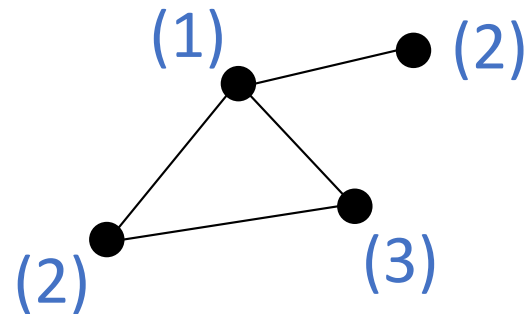
Breadth-First Search (BFS)

```
mark origin as visited;
vertexQueue.enqueue(origin);
while(!vertexQueue.isEmpty()) {
    frontVertex = vertexQueue.dequeue();
    while(frontVertex has a neighbor) {
        nextNeighbor = next neighbor of frontVertex;
        if(nextNeighbor is not visited) {
            mark nextNeighbor as visited;
            vertexQueue.enqueue(nextNeighbor);
        }
    }
}
```

How can this be used to find a path
from origin to a specific other vertex t?

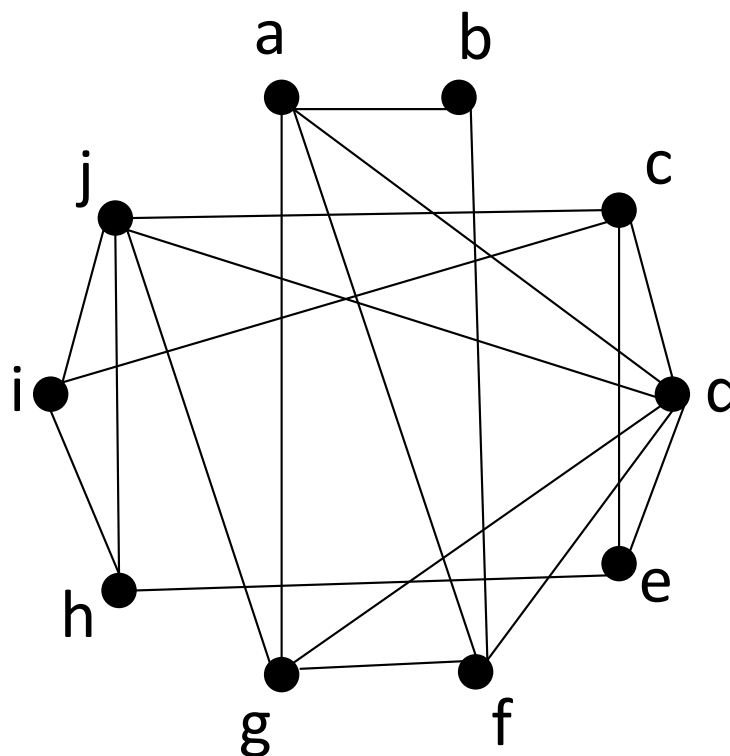
Graph coloring

- Assign each vertex a “color” so that no pair of adjacent vertices have the same color
 - Colors are typically numbers 1, 2, etc
- Goal is to use as few colors as possible



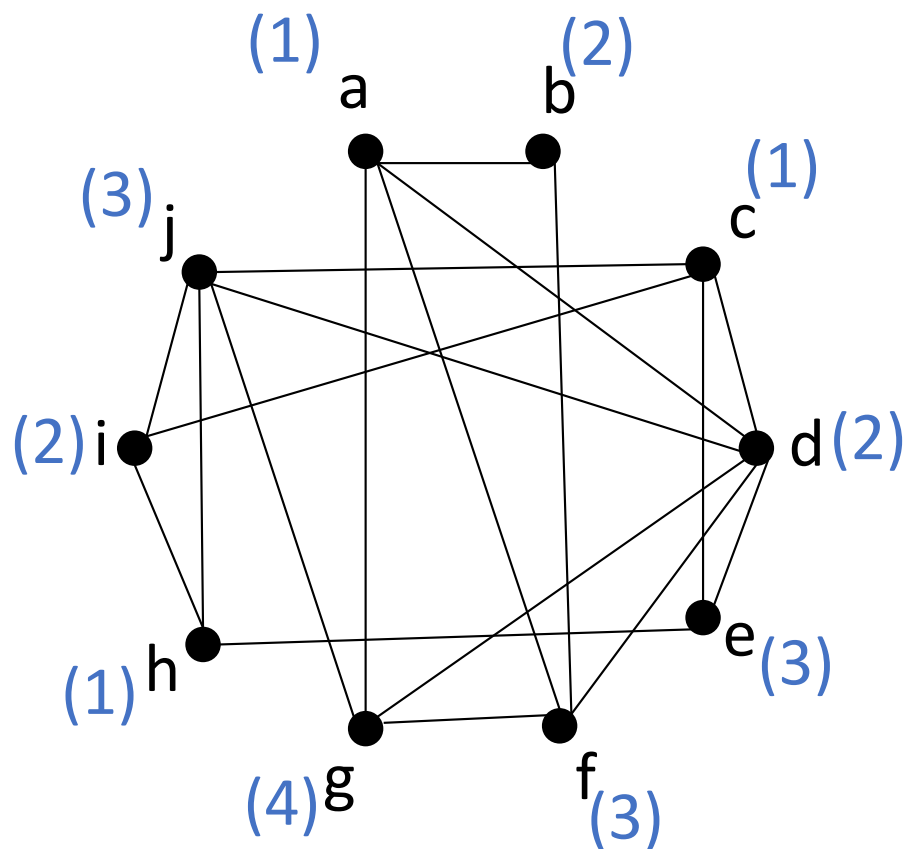
What is the fewest number of colors that can be used to color the graph below?

- A. 2
- B. 3
- C. 4
- D. 5
- E. 6



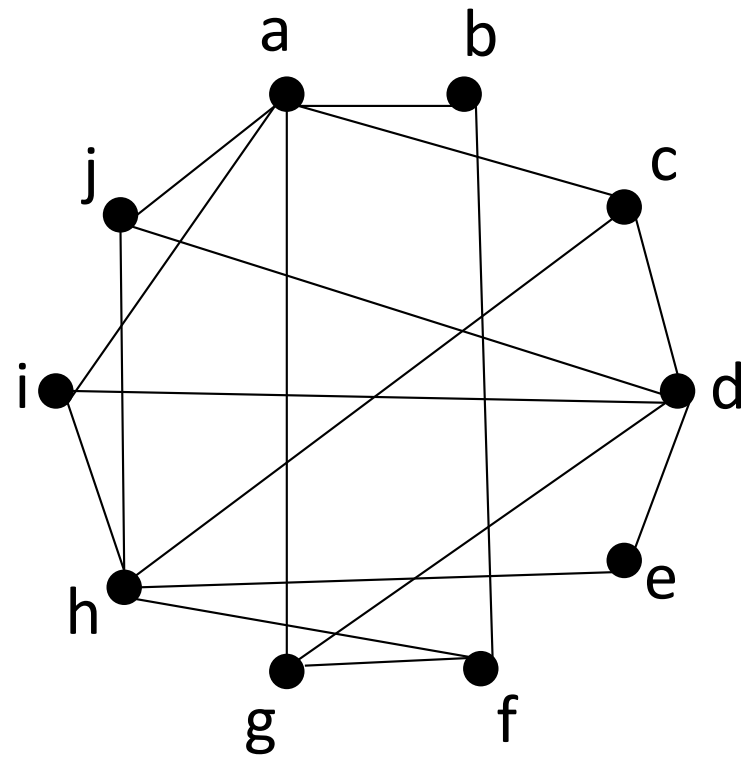
What is the fewest number of colors that can be used to color the graph below?

- A. 2
- B. 3
- C. 4
- D. 5
- E. 6



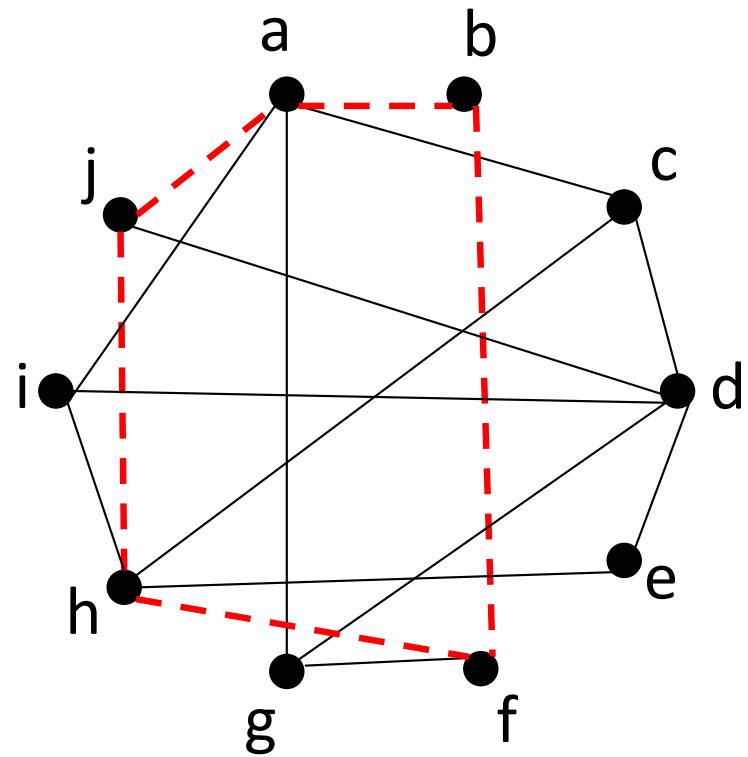
Is this graph 2-colorable?

- A. Yes
- B. No



Is this graph 2-colorable?

- A. Yes
- B. No



Is this graph 2-colorable?

A. Yes

B. No

Give an algorithm to determine
if a graph is 2-colorable

